

Question 2. Build APIs Using Node Js

Requirement: Create a Node.js project in Visual Studio Code, create server.js, and configure Express, body-parser, and MySQL.

```
1 const express = require('express');
2 const bodyParser = require('body-parser');
3 const mysql = require('mysql2');
4
5 // Initialize the Express application
6 const app = express();
7 const port = Number(process.env.PORT) || 3000;
8
9 // Use bodyParser to parse incoming request bodies as JSON
10 app.use(bodyParser.json());
11
12 // MySQL database connection configuration
13 const db = mysql.createConnection({
14   host: process.env.DB_HOST || '127.0.0.1',
15   user: process.env.DB_USER || 'root',
16   password: process.env.DB_PASSWORD || '',
17   database: process.env.DB_NAME || 'nodejs_demo'
18 });
19
20 // Connect to the MySQL database
21 db.connect((err) => {
22   if (err) {
23     console.error('Error connecting to the database:', err);
24     return;
25   }
26   console.log('Connected to the MySQL database');
27 });
```

Requirement: Implement the Create, Get All, and Get by ID user API endpoints.

```
28 // 1. Create a new user (POST request)
29 app.post('/api/users', (req, res) => {
30   const { name, email } = req.body;
31   if (!name || !email) {
32     return res.status(400).json({ message: 'Name and email are
required' });
33   }
34   const sql = 'INSERT INTO users (name, email) VALUES (?, ?) ';
35
36   db.query(sql, [name, email], (err, result) => {
37     if (err) {
38       res.status(500).json({ message: 'Error creating user',
error: err });
39     } else {
40       res.status(201).json({ message: 'User created', userId:
result.insertId });
41     }
42   });
43
44 });
45
46
47 // 2. Get all users (GET request)
48 app.get('/api/users', (req, res) => {
49   const sql = 'SELECT * FROM users';
50
51   db.query(sql, (err, results) => {
52     if (err) {
53       res.status(500).json({ message: 'Error fetching users',
error: err });
54     } else {
55       res.json(results);
56     }
57   });
58 });
59
60 // 3. Get a single user by ID (GET request)
61 app.get('/api/users/:id', (req, res) => {
62   const userId = req.params.id;
63   const sql = 'SELECT * FROM users WHERE id = ?';
64
65   db.query(sql, [userId], (err, results) => {
66     if (err) {
67       res.status(500).json({ message: 'Error fetching user',
error: err });
68     } else if (results.length === 0) {
69       res.status(404).json({ message: 'User not found' });
70     } else {
71       res.json(results[0]);

```

Requirement: Implement the Update and Delete user API endpoints.

```
75
76 // 4. Update a user by ID (PUT request)
77 app.put('/api/users/:id', (req, res) => {
78     const userId = req.params.id;
79     const { name, email } = req.body;
80     if (!name || !email) {
81         return res.status(400).json({ message: 'Name and email are
            required' });
82     }
83     const sql = 'UPDATE users SET name = ?, email = ? WHERE id = ?';
84
85     db.query(sql, [name, email, userId], (err, result) => {
86         if (err) {
87             res.status(500).json({ message: 'Error updating user',
            error: err });
88         } else if (result.affectedRows === 0) {
89             res.status(404).json({ message: 'User not found' });
90         } else {
91             res.json({ message: 'User updated' });
92         }
93     });
94 });
95
96 // 5. Delete a user by ID (DELETE request)
97 app.delete('/api/users/:id', (req, res) => {
98     const userId = req.params.id;
99     const sql = 'DELETE FROM users WHERE id = ?';
100
101     db.query(sql, [userId], (err, result) => {
102         if (err) {
103             res.status(500).json({ message: 'Error deleting user',
            error: err });
104         } else if (result.affectedRows === 0) {
105             res.status(404).json({ message: 'User not found' });
106         } else {
107             res.json({ message: 'User deleted' });
108         }
109     });
110 });
```

Requirement: Run the project with Node server.js or npm start and verify the CRUD API against the MySQL users table.

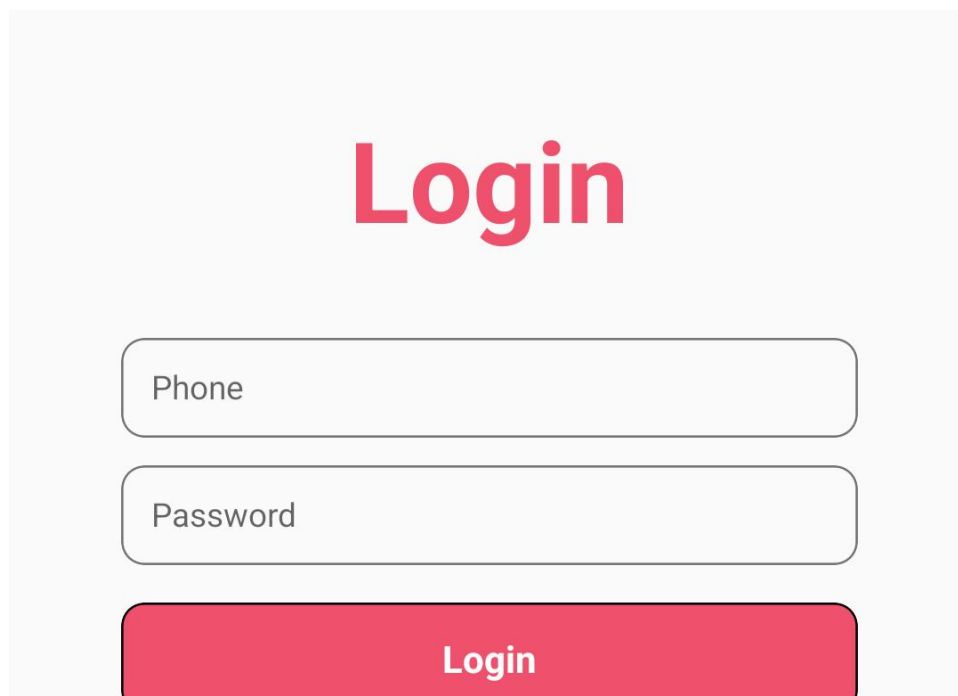
```
POST /api/users      -> 201 Created (user id 2)
GET  /api/users/2    -> 200 User returned
PUT  /api/users/2    -> 200 User updated
DELETE /api/users/2  -> 200 User deleted
Database: local MySQL nodejs_demo.users
```

Question 3. Login Screen Interface Design

Requirement: Create the Q2Login project, install `@react-navigation/native`, create `Styles.js` and `Login.js`, and render the Login component from `App.tsx`.

```
1 import { View, Text, TextInput, ScrollView, TouchableOpacity } from
  'react-native';
2 import styles from './Styles'
3 const LoginScreen = () => {
4   return (
5     <ScrollView showVerticalScrollIndicator={false}>
6       <View style={styles.container}>
7         <Text style={styles.title}>
8           Login
9         </Text>
10        <TextInput style={styles.input} placeholder="Phone" />
11        <TextInput
12          style={styles.input}
13          placeholder="Password"
14          secureTextEntry
15        />
16
17        <TouchableOpacity style={styles.button}>
18          <Text style={styles.buttonText}>Login</Text>
19        </TouchableOpacity>
20      </View>
21    </ScrollView>
22  )
23 };
24
25 export default LoginScreen;
```

Requirement: Run the React Native app on Android. The login screen provides Phone and Password inputs and a Login button.



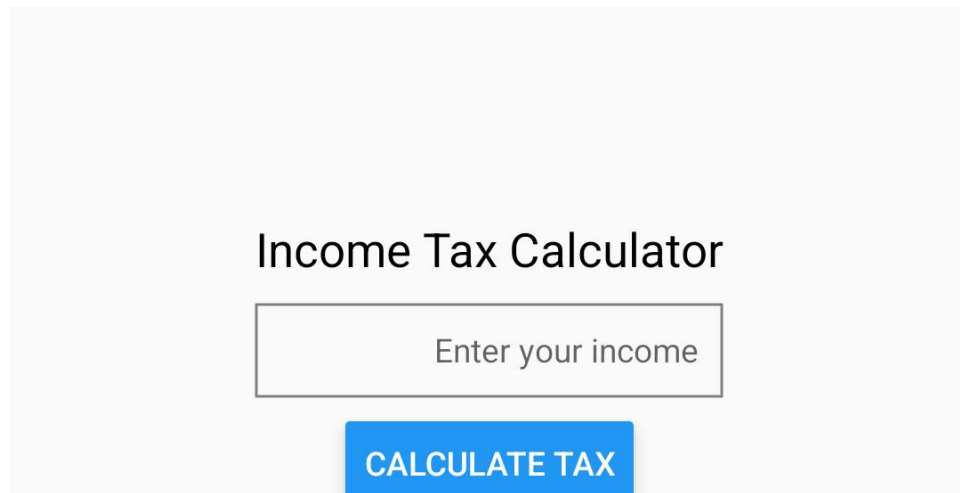
The image shows a mobile app login screen. At the top, the word "Login" is displayed in a large, bold, pink font. Below the title, there are two input fields: the first is labeled "Phone" and the second is labeled "Password". Both fields have a light gray background and a thin gray border. At the bottom of the screen, there is a large, rounded rectangular button with a solid pink background and the word "Login" in white text.

Question 4. Personal Income Tax Calculation

Requirement: Create the Q3IncomeTax project and styles.js, then implement the income-tax calculation in App.tsx using the required 10%, 20%, and 30% brackets.

```
7 import React, { useState } from 'react';
8 import { View, Text, TextInput, Button } from 'react-native';
9 import styles from './styles';
10
11 const App = () => {
12   const [income, setIncome] = useState('');
13   const [tax, setTax] = useState('');
14
15   const calculateTax = () => {
16     const incomeAmount = parseFloat(income);
17
18     if (isNaN(incomeAmount) || incomeAmount < 0) {
19       setTax('Invalid income');
20       return;
21     }
22
23     let taxAmount = 0;
24     if (incomeAmount <= 10000000) {
25       taxAmount = incomeAmount * 0.1;
26     } else if (incomeAmount <= 50000000) {
27       taxAmount = 10000000 * 0.1 + (incomeAmount - 10000000) * 0.2;
28     } else {
29       taxAmount =
30         10000000 * 0.1 + 40000000 * 0.2 + (incomeAmount - 50000000) *
31         0.3;
32     }
33     setTax(`Income Tax: ${taxAmount.toFixed(2)}d`);
34   };
35   return (
36     <View style={styles.container}>
37       <Text style={styles.title}>Income Tax Calculator</Text>
38       <TextInput
39         textAlign="right"
40         style={styles.input}
41         placeholder="Enter your income"
42         keyboardType="numeric"
43         value={income}
44         onChangeText={text => setIncome(text)}
45       />
46       <Button title="Calculate Tax" onPress={calculateTax} />
47       <Text style={styles.result}>{tax}</Text>
48     </View>
49   );
50 };
51
52 export default App;
```

Requirement: Run the application on Android and provide an income input with a Calculate Tax button and result area.

The image shows a mobile application interface for an "Income Tax Calculator". It features a light gray background. At the top, the title "Income Tax Calculator" is centered in a large, black, sans-serif font. Below the title is a white rectangular input field with a thin gray border, containing the placeholder text "Enter your income" in a gray font. Directly beneath the input field is a solid blue rectangular button with the text "CALCULATE TAX" in white, uppercase, sans-serif font.

Income Tax Calculator

CALCULATE TAX

Question 5. Calculator Screen Interface Design

Requirement: Create the Q3Calculator project and style.js. Use state for displayValue, operator, and firstValue, then implement number, operator, equal, and clear handlers.

```
10 function App() {
11   const [displayValue, setDisplayValue] = useState('0');
12   const [operator, setOperator] = useState<string | null>(null);
13   const [firstValue, setFirstValue] = useState('');
14
15   const handleNumberInput = (num: number) => {
16     if (displayValue === '0') {
17       setDisplayValue(num.toString());
18     } else {
19       setDisplayValue(displayValue + num);
20     }
21   };
22
23   const handleOperatorInput = (op: string) => {
24     setOperator(op);
25     setFirstValue(displayValue);
26     setDisplayValue('0');
27   };
28
29   const handleEqual = () => {
30     const num1 = parseFloat(firstValue);
31     const num2 = parseFloat(displayValue);
32     let result: number | string = 0;
33
34     if (operator === '+') result = num1 + num2;
35     else if (operator === '-') {
36       result = num1 - num2;
37     } else if (operator === '*') {
38       result = num1 * num2;
39     } else if (operator === '/') {
40       result = num2 !== 0 ? num1 / num2 : 'Error';
41     }
42
43     setDisplayValue(result.toString());
44     setOperator(null);
45     setFirstValue('');
46   };
47
48   const handleClear = () => {
49     setDisplayValue('0');
50     setOperator(null);
51     setFirstValue('');
```

Requirement: Create the calculator interface with number buttons and arithmetic operators connected to the event handlers.

```
53 return (
54   <View style={styles.container}>
55     <Text style={styles.display}>{displayValue}</Text>
56
57     <View style={styles.row}>
58       {[7, 8, 9].map(num => (
59         <TouchableOpacity
60           key={num}
61           style={styles.button}
62           onPress={() => handleNumberInput(num)}
63         >
64           <Text style={styles.buttonText}>{num}</Text>
65         </TouchableOpacity>
66       )]}
67       <TouchableOpacity
68         style={styles.buttonOperator}
69         onPress={() => handleOperatorInput('/')}>
70       >
71         <Text style={styles.operatorText}>÷</Text>
72       </TouchableOpacity>
73     </View>
74
75     <View style={styles.row}>
76       {[4, 5, 6].map(num => (
77         <TouchableOpacity
78           key={num}
79           style={styles.button}
80           onPress={() => handleNumberInput(num)}
81         >
82           <Text style={styles.buttonText}>{num}</Text>
83         </TouchableOpacity>
84       )]}
85       <TouchableOpacity
86         style={styles.buttonOperator}
87         onPress={() => handleOperatorInput('*')}>
88       >
89         <Text style={styles.operatorText}>x</Text>
90       </TouchableOpacity>
```


Requirement: Add the final calculator row with 0, +, Equals, and Clear actions.

```
93     <View style={styles.row}>
94       {[1, 2, 3].map(num => (
95         <TouchableOpacity
96           key={num}
97           style={styles.button}
98           onPress={() => handleNumberInput(num)}
99         >
100           <Text style={styles.buttonText}>{num}</Text>
101         </TouchableOpacity>
102       )]}
103     <TouchableOpacity
104       style={styles.buttonOperator}
105       onPress={() => handleOperatorInput('-')}
106     >
107       <Text style={styles.operatorText}>-</Text>
108     </TouchableOpacity>
109   </View>
110
111   <View style={styles.row}>
112     <TouchableOpacity
113       style={styles.zeroButton}
114       onPress={() => handleNumberInput(0)}
115     >
116       <Text style={styles.buttonText}>0</Text>
117     </TouchableOpacity>
118     <TouchableOpacity
119       style={styles.plusOperator}
120       onPress={() => handleOperatorInput('+')}
121     >
122       <Text style={styles.plusOperatorText}>+</Text>
123     </TouchableOpacity>
124     <TouchableOpacity style={styles.equalButton}
125       onPress={handleEqual}>
126       <Text style={styles.equalText}>=</Text>
127     </TouchableOpacity>
128   </View>
129   <TouchableOpacity style={styles.clearButton}
130     onPress={handleClear}>
131     <Text style={styles.clearText}>C</Text>
132   </TouchableOpacity>
133 </View>
134 );
135 }
136 export default App;
```